

Formalization of Gödel’s Incompleteness Theorems in Basic Recursive Arithmetic (BRA)

Summary of an Agda development

May 2026

Abstract

We present a fully formalized proof of both Gödel’s First and Second Incompleteness Theorems in a tree-based variant of Guard’s Basic Recursive Arithmetic (BRA), executed in Agda 2.8 with `--safe --without-K --exact-split`. The development contains **zero postulates and zero holes**; `godelIII : Deriv Con -> Deriv bot` typechecks end-to-end against the abstract Goedel II skeleton.

There is a single `Term` datatype (no separate `Tree` datatype), and “codes” are exactly the value-shaped Terms certified by an `IsValue : Term -> Set` predicate (built only from `O` and `ap2 Pair`). The map `reify : Tree -> Term`, written explicitly throughout the codebase for clarity at code/term boundaries, is the identity.

The headline results are:

- $thm11 : Deriv\ G \rightarrow Deriv\ \perp$ (Gödel I);
- $godelIII : Deriv\ Con \rightarrow Deriv\ \perp$ (Gödel II).

The development follows Guard’s 1963 lecture notes [1]. The intermediate construction is a single first-order combinator `constr5 : Fun1` realising Guard’s “ $F(x)$ ” (page 17); each step is given by an explicit equational derivation.

A key conceptual point this note emphasises (Section 5) is that intermediate steps of Theorem 14’s chain produce *equational* `Derivs` of the form `atomic (eqn (thmT t) u)` where `u` is *not* `codeFormula P` for any $P : Formula$. Only the final `step5` conclusion has `u = codeFormula bot` (with $\perp$ genuinely closed). Guard’s chain operates one level lower than Hilbert–Bernays–Löb: instead of internal provability claims about specific formulas, it manipulates substituted-codes through standard BRA inference rules. This makes the entire proof internal and equational — a feature of the framework, not a gap.

1 Introduction

The aim is a feasible, ASCII-only, syntax-first Agda formalization of Gödel’s two incompleteness theorems following Guard’s 1963 system of *Basic Recursive Arithmetic* [1].

1.1 Single-layer Term language with *IsValue*

In BRA2 there is one syntactic layer.

The Term language.

- *Term*: built from `O` (leaf), `var n`, `ap1 f t`, `ap2 g t1 t2`.

- Function symbols: Fun_1 ($\{I, Fst, Snd, Comp, Comp_2, Z\}$, plus $KT_$ as a defined function), Fun_2 ($\{Pair, Const, Lift, Post, Fan, IfLf, TreeEq, treeRec\}$).
- *Formula*: $atomic (eqn \ell r)$, $not _$, $_imp _$.
- *Deriv F*: inductive family of derivations. Constructors for the algebraic axioms (ax_*), the equality axioms (Guard’s Ax 4–7), three propositional axioms (axK , axS , $axNeg$ in Lukasiewicz form), modus ponens (mp), substitution ($ruleInst$), and binary tree induction ($ruleIndBT$).

Codes are value-shaped Terms. “Codes” are Terms whose value-shape is certified by

$$IsValue : Term \rightarrow Set,$$

with constructors $valO : IsValue O$ and $valNd a b v_a v_b : IsValue (ap_2 Pair a b)$ (when $IsValue a$ and $IsValue b$).

The encoding maps now have type

$$code : Term \rightarrow Term, \quad codeFormula : Formula \rightarrow Term,$$

together with structural-soundness lemmas $code_isValue : (t : Term) \rightarrow IsValue (code t)$, $codeFormula_isValue$, $codeF_1_isValue$, $codeF_2_isValue$, and $natCode_isValue$ (BRA2/Term.agda, BRA2/Formula.agda). Every primitive constructor of an encoding produces a value-shaped Term.

Notational aliases Tree, lf, nd, reify. For continuity with Guard’s prose, BRA2/Term.agda exposes

$$Tree := Term, \quad lf := O, \quad nd a b := ap_2 Pair a b, \quad reify t := t.$$

The Layer-1/Layer-2 distinction is recorded as the $IsValue$ predicate; syntactically the layers are unified.

Reflection layer (object-level operations on codes). Functors that reason about BRA syntax internally:

- $cor : Fun_1$: defined as $Rec stepCor$ (BRA2/Cor.agda). Spec: $corOfReify : (t : Term) \rightarrow IsValue t \rightarrow Deriv (ap_1 cor t = code t)$. Operationally, cor takes a value-shaped Term (= a “numeral”) and returns its code.
- $sub : Fun_2$, $subT : Fun_2$: coded substitution (BRA2/Sub.agda, BRA2/SubT.agda), now indexed by $IsValue$ on the substituent.
- $thmT : Fun_1$: the proof-checker (Section 2).

Reflection theorem.

$$encode : (P : Formula) \rightarrow Deriv P \rightarrow \Sigma Term (\lambda y. \Sigma (IsValue y) (\lambda _. Deriv (thmT y = codeFormula P))).$$

The $IsValue y$ field certifies that every encoded proof tree is value-shaped, so $cor y$ provably reduces to $code y$ at the witness returned by $encode$. This certification is exactly what discharges the parametric $\mathbf{cor} \ x$ in the chain at the closure (see Section 5).

1.2 Notational convention: code-quotes

We use $\ulcorner E \urcorner$ throughout for “the code (value-shaped Term) of E ”:

$$\ulcorner t \urcorner := \text{code } t \quad (t : \text{Term}), \quad \ulcorner F \urcorner := \text{codeFormula } F \quad (F : \text{Formula}).$$

Whenever a Guard-style expression mixes object-level and encoded material (e.g. $f(\underline{x})$), we keep the code-quote explicit: the formal object is the Term $\ulcorner f(\underline{x}) = \underline{f(x)} \urcorner$, never the unquoted equation. See Section 4.1 for the full caveat.

2 The provability predicate $\text{thm}T$

The Gödel-style “ th ” provability predicate is realized in BRA by a single Fun_1 -functor:

$$\text{thm}T = \text{Rec stepProtoWrapped} : \text{Fun}_1.$$

Here stepProtoWrapped is a Fun_2 assembled from a 40-deep linear IfLf cascade (BRA2/Thm/StepProto.agda and BRA2/Thm/ThmT.agda), one branch per primitive proof rule: ten equational axioms of Group I, eight Group II axioms, nine equality and propositional rules, and three rules of inference (mp , ruleInst , ruleIndBT).

The *key* property of $\text{thm}T$ is the encoding-soundness lemma *encode* above, proved by induction on the *Deriv* derivation in BRA2/Thm/ThmT.agda’s *encodeRich*.

2.1 Self-substitution closure

$\text{thm}T$ is a closed combinator:

$$\text{thClosed} : \forall x \, r, \quad \text{substF}_1 \, x \, r \, \text{thm}T = \text{thm}T$$

and its encoding is var-free:

$$\text{codeF1Th_noVar} : \forall x \, \text{repl}, \quad \text{codedSubst } \text{repl} \, (\text{code } (\text{var } x)) \, (\text{codeF}_1 \, \text{thm}T) = \text{codeF}_1 \, \text{thm}T.$$

3 Gödel’s First Incompleteness Theorem (Theorem 11)

3.1 The diagonal sentence

Following Guard, define

$$\begin{aligned} F_{\text{pre}} &:= \neg(\text{thm}T \, v_1 = \text{sub } v_0 \, v_0), \\ j &:= \text{codeFormula } F_{\text{pre}} \quad (\text{a closed value-shaped Term}), \\ i &:= \text{reify } j = j, \\ G &:= F_{\text{pre}}[v_0 := i]. \end{aligned}$$

The sentence G has v_1 free; informally, G asserts of itself that no proof tree v_1 proves the formula whose code is $j[v_0 := i]$.

The certificate $j_isValue : \text{IsValue } j$ is exposed inside the **Diagonal** abstract block (BRA2/Thm11Diagonal.agda). It propagates through *Outer.GodelIII* and is what makes `corOfReify j j_isValue` usable downstream.

3.2 Theorem 11

Theorem 1 (`thm11`, Gödel I). $thm11 : Deriv\ G \rightarrow Deriv\ \perp$, where $\perp := atomic\ (eqn\ trueT\ falseT)$, $trueT := O$, $falseT := ap_2\ Pair\ O\ O$.

Sketch. From any $\pi : Deriv\ G$ extract its encoding (y_d, v_d, E_1) where $v_d : IsValue\ y_d$ and $E_1 : Deriv\ (thmT\ y_d = codeFormula\ G)$. A separate diagonal lemma (`BRA2/Thm11Diagonal.agda`) yields $N : Deriv\ \neg(thmT\ y_d = codeFormula\ G)$, and $ax_{ExFalse} + two\ mp$'s collapse E_1 and N to $Deriv\ \perp$. $\square$

4 Gödel's Second Incompleteness Theorem (Theorem 14)

Theorem 2 (`godelIII`, Gödel II). $godelIII : Deriv\ Con \rightarrow Deriv\ \perp$, where $Con := \neg(thmT\ v_0 = codeFormula\ \perp)$.

4.1 Caveat: “ $f(\underline{x}) = \underline{f(x)}$ ” is not a BRA formula

A structurally crucial point, easy to miss in Guard's notation: the expression

$$f(\underline{x}) = \underline{f(x)}$$

is **not** a BRA formula. Guard's underline $(\underline{\cdot})$ takes a value and returns its numeral term — well-defined only at the meta-level for closed numerical inputs. For a variable x (the case Theorems 12–14 actually need), neither $\underline{x}$ nor $\underline{f(x)}$ has any BRA referent: there is no operator in BRA's term language that, given an arbitrary Term, returns “the numeral of its value.”

The only formal object that *does* correspond to Guard's expression is its *code*, the Term

$$\ulcorner f(\underline{x}) = \underline{f(x)} \urcorner,$$

which Definition 12 of Guard's notes *defines* by stipulation. In BRA2 this Term is built parametrically as $codeFTeq_1\ f\ x$, with $cor\ x$ filling the slot for the code of x 's numeral and $cor\ (f\ x)$ filling the slot for the code of $f(x)$'s numeral.

When x is a free Term variable, neither $cor\ x$ nor $cor\ (f\ x)$ reduces (the defining equation $corOfReify$ fires only on $IsValue$ inputs), so $codeFTeq_1\ f\ x$ contains ap_1 -headed subterms literally. Codes (in the sense of $codeFormula\ P$) are $IsValue$ -shape, so $codeFTeq_1\ f\ x$ at variable x is **not** of the form $codeFormula\ P$ for any $P : Formula$.

4.2 Theorem 12 — the encoded “ $th\ f$ ” clause

For every Fun_1 functor f , Theorem 12 produces a $Fun_1\ D_f$ such that $thmT$ evaluated at $D_f\ x$ is provably equal to the encoded form of “ $f(x) = \underline{f(x)}$ ”:

Theorem 3 ($thm12_F_1$). $thm12_F_1 : (f : Fun_1) \rightarrow Thm12_F_1_Result\ f$ where $Thm12_F_1_Result\ f$ packages $D_f : Fun_1$ with $proof : (x : Term) \rightarrow Deriv\ (thmT\ (D_f\ x) = codeFTeq_1\ f\ x)$.

The conclusion's RHS is the literal Pair-shape encoded equation. An analogous statement for Fun_2 . Structural recursion on f with one clause per constructor (15 total), `BRA2/Thm12.agda` module *FromBridges*.

4.3 Theorem 13 — under hypothesis $f\ x = y$

Lemma 4 ($thm13_F_1$). For every $f : Fun_1$ and every Theorem 12 result r_{12} for f , $thm13_F_1\ f\ r_{12}\ x\ y : Deriv\ (f\ x = y) \rightarrow Deriv\ (thmT\ (D_f\ x) = codeFTeq_{1,Hyp}\ f\ x\ y)$.

The conclusion replaces $cor\ (f\ x)$ by $cor\ y$. One-line proof: *ruleTrans* on $r_{12}.proof\ x$ with *cong_R Pair* on *cong₁ cor* of the hypothesis.

4.4 Theorem 14 — the consistency-to-bot construction

The contract abstracted in BRA2/Thm14Abstract.agda asks for:

1. a Fun_1 -functor *constr5* such that $thmT\ (constr5\ x)$ syntactically encodes $\perp$;
2. a parametric internal-implication

$$step5 : (x : Term) \rightarrow Deriv((thmT\ x = codeFormula\ G) \text{ imp } (thmT\ (constr5\ x) = codeFormula\ \perp)).$$

The closure *closureToG* instantiates *step5* at v_1 , *ruleInsts Con*, modus-ponenses, transports through *subIIeqcG* and G_{unfold} , arriving at $Deriv\ G$. Composing with *thm11* gives *godellI*.

4.4.1 Construction of *constr5*

Two pre-existing tautologies, encoded:

- $t := (v_0 = v_2) \text{ imp } ((v_1 = v_2) \text{ imp } (v_0 = v_1))$ (transitivity);
- $t' := (v_0 = v_1) \text{ imp } ((\neg(v_0 = v_1)) \text{ imp } \perp)$ (ex falso).

Both BRA-derivable (BRA2/Thm14Numerals.agda); *encode* yields value-shaped Terms *tterm*, *t'term* certified by *t_term.isValue* / *t'_term.isValue* (these certificates are load-bearing for the chain — see Section 5).

The chain, paralleling Guard's page 17:

$$\begin{aligned} f_part(x) &:= ruleInst\ v_0\ (th\ \hat{x})\ (ruleInst\ v_1\ (sub\ i\ i)\ (ruleInst\ v_2\ cor\ G\ tterm)), \\ g_part(x) &:= mp(mp(f_part(x), D_{thmT}\ x), D_{sub}\ i\ i), \\ K_part(x) &:= ruleInst\ v_1\ (cor\ x)\ x, \\ h_part_skel(x) &:= ruleInst\ v_0\ (th\ \hat{x})\ (ruleInst\ v_1\ (sub\ i\ i)\ t'term), \\ constr5_{final}(x) &:= mp(mp(h_part_skel(x), g_part(x)), K_part(x)). \end{aligned}$$

4.4.2 The five steps

(BRA2/Th14*.agda.)

Step 1 — *step1_l* Theorem 13 applied to *thmT*.

Step 2 — *step2_l* Theorem 13 applied to *sub* at $(i, i, cor\ G)$ under closed hypothesis *subIIeqcG*.

Step 3 — *step3_l* Three sb-applications on *t*, canonicalised, then two *mp* dispatches.

Step 4 — *K_part_l* One sb-application at v_1 on G_{norm} via the *PrAtX x* hypothesis to swap $thmT\ x \mapsto cor\ G$.

Step 5 — *step5_l* Two outer *mp* dispatches at $constr5_{inner,final}$ and $constr5_{final}$.

5 What is going on at the encoded layer: a remarkable internal proof

5.1 The phenomenon

The chain $step1_l, \dots, step5_l$ proves *Derivs* of the schematic form

$$Deriv \left(PrAtX \ x \ imp \ atomic \ (eqn \ (thmT \ t_i) \ u_i) \right),$$

where t_i, u_i are Terms depending on x . Several of these intermediate u_i are **not** of the form *codeFormula P* for any $P : Formula$.

A representative example: $step3_l$ (BRA2/Th14Step3.agda) concludes

$$Deriv \left(PrAtX \ x \ imp \ atomic \ (eqn \ (thmT \ (g_part \ x)) \ ap_2 \ Pair \ (\widehat{th \ \hat{x}} \ \widehat{sub \ i \ i})) \right),$$

with

$$\widehat{th \ \hat{x}} = ap_2 \ Pair \ tagAp_1 \ (ap_2 \ Pair \ (codeF_1 \ thmT) \ (ap_1 \ cor \ x)).$$

At a variable x (e.g. $x = v_1$), $ap_1 \ cor \ x$ does not reduce: *corOfReify* requires *IsValue* x , and v_1 is not value-shape. The Term $ap_1 \ cor \ x$ has an ap_1 head; codes are exclusively *IsValue*-shape (built from O and $ap_2 \ Pair$). Therefore $u_3 = ap_2 \ Pair \ (\widehat{th \ \hat{x}} \ \widehat{sub \ i \ i})$ is not in the image of *codeFormula*.

The same phenomenon recurs in $step4, step5_{intermediate}, h_part_skel_l$. In each case, u_i is a *substituted-code* — the result of inserting $cor \ x$ into a variable slot of the code of an open formula.

Yet *Deriv (atomic (eqn t u))* is perfectly well-formed for arbitrary Terms t, u — the equational predicate *eqn* does not require u to be a code of a formula. The chain composes via the standard inference rules (*mp*, *ruleTrans*, *cong**, *axS*, *axK*) without ever needing the intermediate u_i to be *codeFormula P*.

5.2 Why this works: substituted-codes vs. codes-of-formulas

There are two distinct readings of Guard's notation $\ulcorner f(\underline{x}) \urcorner = \underline{f(x)}$:

- (A) **Code-of-substituted-formula.** Read $\underline{x}$ as $cor \ x$ (a BRA Term), and form the BRA formula *atomic (eqn (ap₁ f (cor x)) ...)*. Take its *codeFormula*. The inner slot becomes *code (cor x)* (= the code of the BRA term $cor \ x$).
- (B) **Substituted-code.** Take the open formula *atomic (eqn (ap₁ f v_n) ...)*, code it, then *sb*-substitute $cor \ x$ into the $\ulcorner v_n \urcorner$ slot. The inner slot becomes $cor \ x$ itself.

At a free variable x these readings disagree: (A) places *code (cor x)* in the slot, (B) places $cor \ x$. Theorem 12's *codeFTeq₁* uses reading (B) — the substituted-code form. This is forced by the structure of D_f , which is built from *Comp₂ Pair ... cor*, producing $cor \ x$ in its evaluation directly.

The two readings coincide **when x is value-shaped**: *corOfReify* then gives $cor \ x = code \ x$, and *Lemma codeCommutesFormula* (BRA2/CodeCommutes.agda) gives

$$code \ (substF \ n \ t \ P) = codedSubst \ (code \ t) \ (code \ (var \ n)) \ (codeFormula \ P).$$

So at value-shape x , the substituted-code (B) is BRA-equal to *codeFormula (substF n x P_{open})* — a genuine code-of-formula. At variable x this collapse fails, and (B) remains a hybrid Term.

5.3 The chain’s discharge mechanism

Why does Goedel II close even though step 3 etc. produce hybrid RHS?

1. *Internal composition stays at the equational level.* The chain’s `mp/cong/ruleTrans` steps operate on `eqn t u` for arbitrary t, u ; they do not require u to be `codeFormula P`. All intermediate Derivs are honest BRA derivations producing well-typed atomic equations.
2. *step5’s conclusion is genuinely codeFormula $\perp$.* At step 5 the final RHS is `codeFormula  $\perp$` , where $\perp$ is closed. No `cor x` remains in the conclusion — the `Comp2-of-KT-of-codeFormula-bot` construction in `constr5final` is a constant in x modulo Lift, so its `thmT`-image is a closed code by construction.
3. *Closure transports through notEqTransport at a value-shape witness.* `closureToG` instantiates `step5` via `ruleInst` at the encode-witness of G ’s proof. By the strengthened `encode` contract (returning `IsValue`), this witness is a value-shaped Term. At such a witness, `cor x` inside any intermediate substituted-code rewrites to `code x` via `corOfReify`, and the substituted-codes collapse to genuine `codeFormulas` of substituted formulas.

The interleaving — carry hybrid forms through internal-implication chains, collapse at the closure — is what lets the proof be parametric over $(x : \text{Term})$ rather than meta-iterated over closed numerals.

5.4 Comparison with Hilbert–Bernays–Löb

Standard textbook Gödel II proofs go through provability statements:

$$D1 : \vdash A \implies \vdash \text{Pr}(\ulcorner A \urcorner), \quad D2 : \vdash \text{Pr}(\ulcorner A \rightarrow B \urcorner) \rightarrow \text{Pr}(\ulcorner A \urcorner) \rightarrow \text{Pr}(\ulcorner B \urcorner), \quad D3 : \vdash \text{Pr}(\ulcorner A \urcorner) \rightarrow \text{Pr}(\ulcorner \text{Pr}(\ulcorner A \urcorner) \urcorner).$$

Every premise is a provability statement *about* a code of a specific formula. The chain stays at the level “ F is provable” for various F . Reflection (D3) is needed to handle the iteration.

Guard’s BRA chain operates one level lower: at the level of *equations* `thmT t = u` for arbitrary Terms t, u . Reflection (D3-analogue) is realised internally and parametrically, without reference to any specific formula:

- Theorem 12 is the parametric “ D_f is the encoded image of f ” — one statement, all f .
- Theorem 13 lifts external equations to encoded form, parametric in the equational hypothesis.
- Theorem 14 chains substituted-codes through `mp/ sb/ruleInst`, never invoking a separate “ $\text{Pr}(\ulcorner \text{Pr}(\ulcorner G \urcorner) \urcorner)$ ” reflection axiom. `step5` is an equational consequence of Theorem 12’s defining equations under hypothesis, not a provability claim about a specific formula.

5.5 Remarkable consequences

1. **Avoids meta-reflection.** No $\text{Pr}(\ulcorner \text{Pr}(\ulcorner F \urcorner) \urcorner)$ axiom is invoked. The analogous internalisation falls out of `thmT`’s defining equations + `Comp2/KT/cor` compositions.
2. **Fully equational.** Goedel II reduces to BRA equational reasoning + `mp/axS/axK/axNeg/axContrapos`. No specialised “provability logic” is needed.

3. **Parametric over $(x : \text{Term})$.** Each step lemma works for any Term x (variable, application of cor to a variable, anything). Standard HBL proofs are usually meta-iterated over each closed witness.
4. **The intermediate Derivs are genuine BRA proofs that do not correspond to “BRA proves formula F ” for any single F .** This is the structurally interesting feature: the chain proves *equational* statements between Terms, with the “provability” interpretation recovered only at the closure.

The cost is bookkeeping that the development makes explicit: the substituted-code RHS at variable x requires *IsValue* certificates (on j , on encode-witnesses, on t_term/t'_term) propagated through to the closure. These certificates are the formal counterpart of Guard’s silent assumption that “ x ” for variable x is sensible because x will eventually be instantiated at a closed numeral.

In short: *Goedel II in BRA is an equational proof at the code layer, internalised via primitive recursion, with the provability-logic story emerging only at closure.* Guard’s 1963 formulation has this character intrinsically; HBL-style proofs do not. The Agda dev makes the distinction visible.

6 Methodology

6.1 Constraints maintained throughout

- `--safe --without-K --exact-split` on every file.
- ASCII only. No Unicode.
- No postulates. No holes. No `with`-abstraction. No dot patterns (one exception: `.0 / .a .b` in *IsValue* pattern matches, which are the canonical syntax for matching against an indexed family’s index).
- No new *Deriv* axioms beyond Guard’s primitive list.

6.2 Verification

```
$ agda --safe BRA2/GoedelIIIFull.agda
$ echo $?
0
$ ls BRA2/GoedelIIIFull.agdai
BRA2/GoedelIIIFull.agdai
$ grep -rn "^postulate" BRA2/
$ # (empty)
```

7 Discussion

The two Gödel theorems in BRA2:

- **Gödel I.** If the diagonal sentence G is provable, then $\perp$ is provable.
- **Gödel II.** If the consistency formula Con is provable, then $\perp$ is provable.

The intermediate functions $constr5_{\text{final}}$ and the parametric step lemmas $step1_l$ – $step5_l$, K_part_l , $h_part_skel_l$ are *first-order* in the sense of recursive-arithmetic definability: each is a closed combinator built from primitive Fun_1 / Fun_2 symbols, and each *Deriv* derivation is given by an explicit equational proof in the BRA system. No external soundness theorem is invoked.

7.1 Comparison with prior formalisations

The only previous machine formalisation of Gödel’s *Second* Incompleteness Theorem we are aware of is Paulson’s Isabelle/HOL development of hereditarily finite set theory [2]. Other formalisations (O’Connor in Coq, Han in Lean, Carneiro’s metamath work) target Gödel’s *First* theorem only.

Paulson’s development uses HF set theory and full Gödel numbering of Hilbert-style proofs, with encoded substitution treated as an external soundness theorem. The present development by contrast follows Guard [1]: the “provability predicate” $thmT$ is itself a primitive-recursive combinator built from $Rec + Fan + Lift + IfLf$; encoded substitution is performed by an explicit primitive recursive function $sub : Fun_2$; the entire chain stays first-order; and the proof is equational at the encoded layer rather than provability-logical at the formula layer (Section 5).

References

- [1] J. R. Guard, *Lecture notes on recursive arithmetic*, Applied Mathematics Division, Argonne National Laboratory, 1963.
- [2] L. C. Paulson, *A machine-assisted proof of Gödel’s incompleteness theorems for the theory of hereditarily finite sets*, Review of Symbolic Logic, vol. 7 no. 3, 2014, pp. 484–498.
- [3] R. Davies and F. Pfenning, *A modal analysis of staged computation*, Journal of the ACM, vol. 48 no. 3, May 2001, pp. 555–604.